

Express.js

Request & Response

Tudo que você precisa saber sobre req e res

O que são req e res?

💡 Pense numa ligação telefônica. Alguém liga pra você (Request), você atende e responde (Response). No Express, toda vez que alguém acessa uma rota, o framework te entrega esses dois objetos prontos para usar.

```
app.get('/usuarios', (req, res) => {  
  // req = a ligação que chegou  
  // res = sua resposta  
})
```

req — Request

Objeto com tudo que o cliente enviou:
URL, corpo, cabeçalhos, parâmetros.

res — Response

Objeto que você usa para enviar
a resposta de volta ao cliente.

req.params & req.query

req.params

Parâmetros que vêm na URL — identificam um recurso específico.

```
app.get('/usuarios/:id', (req, res) => {  
  console.log(req.params.id) // "42"  
})  
  
// Chamada: GET /usuarios/42
```

req.query

Parâmetros após o ? na URL — usados em filtros e buscas.

```
app.get('/produtos', (req, res) => {  
  console.log(req.query.categoria) // "eletronicos"  
})  
  
// Chamada: GET /produtos?categoria=eletronicos
```

req.body & req.headers

req.body

Corpo da requisição — dados enviados em POST e PUT. Requer `express.json()`.

```
app.use(express.json()) // obrigatório!

app.post('/usuarios', (req, res) => {
  console.log(req.body.nome) // "Lucas"
  console.log(req.body.email) // "lucas@email.com"
})
```



Sem `app.use(express.json())`, o `req.body` chega `undefined`. Erro mais comum de iniciante.

req.headers

```
console.log(req.headers['authorization']) // "Bearer eyJhbGci..."
console.log(req.headers['content-type']) // "application/json"
```

Métodos do res

res.json()

Responde com JSON. O mais usado em APIs.

```
res.json({ nome: 'Lucas', idade: 19 })
```

res.status()

Define o status code. Sempre combine com outro método.

```
res.status(201).json({ mensagem: 'Criado!' })
```

res.send()

Envia texto ou HTML genérico.

```
res.send('Olá, mundo!')
```

res.sendStatus()

Manda só o status code, sem body.

```
res.sendStatus(204)
```

O resultado da operação em um número

São sempre 3 dígitos. O cliente lê esse número antes de qualquer outra coisa.

1xx Informacional	2xx Sucesso ✓	3xx Redirecionamento	4xx Erro do cliente	5xx Erro do servidor
---------------------------------	-----------------------------	------------------------------------	-----------------------------------	------------------------------------

Os mais importantes no dia a dia:

200 OK	201 Created	204 No Content	400 Bad Request	401 Unauthorized	403 Forbidden	404 Not Found	409 Conflict	500 Server Error
------------------	-----------------------	--------------------------	---------------------------	----------------------------	-------------------------	-------------------------	------------------------	----------------------------

GET

Buscando dados

Quais status retornar?

200 OK

Encontrou e retornou os dados com sucesso

404 Not Found

O recurso pedido não existe

400 Bad Request

A requisição está malformada

401 Unauthorized

Não está autenticado

403 Forbidden

Autenticado, mas sem permissão

500 Server Error

Algo quebrou no servidor

```
app.get('/usuarios/:id', (req, res) => {  
  const usuario = buscarUsuario(req.params.id)  
  if (!usuario) {  
    return res.status(404).json({ erro: 'Usuário não encontrado' })  
  }  
  res.status(200).json(usuario)  
})
```

POST

Criando um recurso

Quais status retornar?

201 Created

Recurso criado com sucesso

400 Bad Request

Dados inválidos ou faltando

409 Conflict

Recurso já existe (ex: email duplicado)

401 Unauthorized

Não está autenticado

500 Server Error

Erro ao salvar no banco

```
app.post('/usuarios', (req, res) => {  
  const { nome, email } = req.body  
  if (!nome || !email) {  
    return res.status(400).json({ erro: 'Nome e email são obrigatórios' })  
  }  
  const novoUsuario = criarUsuario(nome, email)  
  res.status(201).json(novoUsuario)  
})
```

Atualizando e removendo

PUT / PATCH & DELETE

PUT / PATCH

200 OK

Atualizado — retorna o recurso atualizado

204 No Content

Atualizado — sem retornar body

404 Not Found

O recurso a atualizar não existe

400 Bad Request

Dados inválidos

DELETE

200 OK

Deletado — retorna confirmação

204 No Content

Deletado — sem body

404 Not Found

O recurso a deletar não existe

403 Forbidden

Sem permissão para deletar

Para levar pra vida

**"Sempre retorne um status code
que diga o que realmente aconteceu."**

O cliente — seja um app, outro serviço, ou um dev testando — toma decisões baseado nesse número.
Um 200 mentiroso numa operação que falhou é um dos bugs mais difíceis de rastrear.

